DIGITAL IMAGE PROCESSING

UNIT-1

INTRODUCTION:

An image is a representation of an object, scene, person, or idea as seen by human eyes. An image can be drawn, photographed, or generated by a machine. An image is formed when light reflected or emitted from objects is captured by the human eye or by a surface such as a mirror, lens, or screen. Images help us understand, interpret, and communicate information about the world around us.

Images can be classified into different types based on how they are formed. A **real image** is formed when light rays meet after reflection or refraction, and it can be projected onto a screen (for example, images formed by a projector or camera lens). A **virtual image** is formed when light rays appear to meet but do not actually converge; such images cannot be projected onto a screen (for example, images seen in a plane mirror).

Images may also be natural, such as those seen by the human eye, or artificial, such as photographs, paintings, and drawings. They play an important role in communication, education, science, art, and everyday life by helping people visualize objects and concepts clearly. In general, an image represents how an object appears through reflection, refraction, or artistic creation, enabling visual perception and understanding.

What is a Digital Image?

Digital Image is a multidimensional function. It is represented as f(x, y, z, λ) where x and y are spatial (plane) coordinates. f(x, y, z, λ) represents three-dimensional (3-D) color image, whereas f(x, y, λ) represents two dimensional (2-D) “color image” and f(x, y) represents the “grayscale image”. If the image f(x, y) takes only two values, either “0” or “1”, it is called a “binary image”. An image is a numerical representation of visual information.

Digital Image Processing (DIP) is the use of computer algorithms to perform operations on digital images to enhance them or to extract useful information. It involves converting images into digital form and applying techniques such as filtering, enhancement, restoration, compression, and analysis to improve image quality or obtain meaningful data.

Digital image processing is widely used in fields such as medical imaging, satellite and remote sensing, computer vision, security systems, and multimedia applications. By manipulating pixel values mathematically, DIP helps in tasks like noise removal, edge detection, object recognition, and image segmentation.

The origin of Digital Image Processing

In the early 1920s Bartlane Cable Picture Transmission System was introduced to transport a picture across Atlantic using specialized printing equipment coded pictures for cable transmission, then reconstructed them at the receiving end. At the receiving end, the image was reproduced on a telegraph printer fitted with typefaces simulating the halftone pattern. The early Bartlane system was capable of coding images in five distinct levels of gray. This capability was increased to 15 in 1929. Though the images were transported through cables in a coded format, these images were not considered as digital images just because digital computers were not used in the creation of the images. The History of digital image processing is tied to the developments and inventions in the field of digital computer systems and related fields such as digital display technology, storage technology, networking and communication technology etc.

The computers grew in power and processing speeds through several generations. The early computers like ENIAC, UNIVAC, EDVAC etc., could process only texts, numerals and some special characters. Computers powerful enough to carry out image processing tasks appeared in 1960s. At the same time period, developments in space studies were on the rise. Works on using computer techniques for improving images from space probe began at Jet Propulsion Laboratory (Pasadena, California, USA) in 1964. The pictures of the moon transmitted by Ranger 7 were processed using a digital computer. The below figure shows the first image of moon taken by Ranger 7 on July 31, 1964 at 9:09Am EDT(Eastern Daylight Time) and processed using a digital computer.

Figure: The first picture of the moon by a U.S. spacecraft *Ranger 7*

In the late 1960s and early 1970s , in parallel with space applications, digital image processing techniques began to be used in medical imaging, remote earth resources observation and astronomy. For medical diagnosis Computer Axial Tomography (CAT) or Computerized Tomography (CT) was invented in 1970s.

From 1960s until present, the field of digital image processing has grown vigorously. DIP is now used in wide range of applications. At the developments in DIP has led to new fields such as machine learning, deep learning and artificial intelligence. In other words, DIP techniques are used in solving problems dealing with machine perception. The expansion of networking and communication bandwidth via internet, enhancement of processing speeds, storage capacity and high quality printing and display devices, introduction of SSD for storage, cloud computing abilities have created unprecedented opportunities for continued growth of DIP. Some of these applications are illustrated in the following section.

Examples of Fields that Use Digital Image Processing

Some of the major fields in which digital image processing is widely used are:

1. Gamma Ray Imaging- Nuclear medicine and astronomical observations.
2. X-Ray imaging – X-rays of body.
3. Ultraviolet Band –Lithography, industrial inspection, microscopy, lasers.
4. Visual And Infrared Band – Remote sensing.
5. Microwave Band – Radar imaging

Gamma-Ray Imaging

Major uses of imaging based on gamma rays include nuclear medicine and astronomical observations. In nuclear medicine, the approach is to inject a patient with a radioactive isotope that emits gamma rays as it decays. Images are produced from the emissions collected by gamma ray detectors.

X-ray Imaging

- X-rays are among the oldest electromagnetic (EM) radiation sources used for imaging.
- Their most common application is medical diagnosis, but they are also used in industry and astronomy.
- Computerized Axial Tomography (CAT) is one of the most important X-ray imaging techniques.
- CAT scans revolutionized medicine in the early 1970s due to their high resolution and 3D imaging ability.
- Each CAT image represents a cross-sectional slice taken perpendicular to the patient's body.
- Multiple slices are combined to form a 3D internal image of the patient.
- The longitudinal resolution of the 3D image depends on the number of slices acquired.

Imaging in the Ultraviolet Band

- Ultraviolet (UV) light has a wide range of applications, including lithography, industrial inspection, microscopy, lasers, biological imaging, and astronomy.

- UV imaging is commonly demonstrated using microscopy and astronomical examples.
- UV light is extensively used in fluorescence microscopy, a rapidly growing field.
- Ultraviolet light itself is invisible to the human eye.

Imaging in the Visible and Infrared Bands

- The visible band of the electromagnetic spectrum is the most familiar to humans.
- Imaging in the visible band has the widest range of applications compared to other bands.
- Due to its close relationship, the infrared band is often used together with visible imaging.
- For this reason, visible and infrared imaging are grouped together for illustration.

Imaging in the Microwave Band

- The main application of microwave-band imaging is radar.
- Imaging radar can collect data anytime and anywhere, independent of weather or lighting conditions.
- Radar waves can penetrate clouds and, under certain conditions, vegetation, ice, and very dry sand.
- Radar is often the only method for exploring inaccessible regions of the Earth.
- Imaging radar operates like a flash camera, providing its own illumination using microwave pulses.
- The radar system illuminates the ground and captures a snapshot image.
- Instead of a lens, radar uses an antenna and digital signal processing to form images.
- Radar images show only the microwave energy reflected back to the radar antenna.

Imaging in the Radio Band

- Major applications of radio-band imaging are in medicine and astronomy, like gamma-ray imaging.
- In medicine, radio waves are used in Magnetic Resonance Imaging (MRI).
- MRI places the patient inside a strong magnetic field.
- Radio waves are transmitted through the body in short pulses.
- Each pulse causes body tissues to emit response radio signals.
- A computer analyzes the origin and strength of these signals.
- The processed data forms a two-dimensional image of a body section.
- MRI can generate images in any plane (axial, sagittal, or coronal).

Fundamental Steps in Digital Image Processing

The field of digital image processing refers to processing digital images by means of a digital computer.

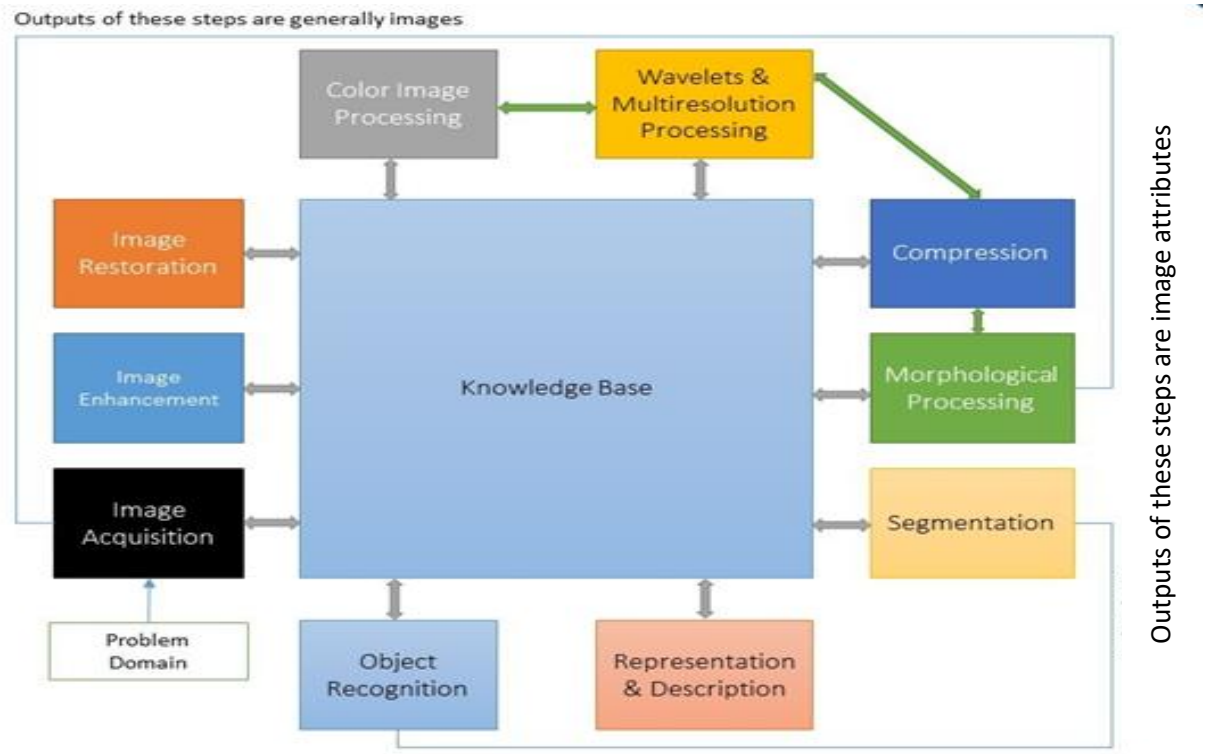


Figure: Steps in Digital Image Processing

1. Image Acquisition: Image acquisition is the process of loading digital images.
2. Image Enhancement: Image enhancement is among the simplest and most appealing areas of digital image processing, such as, changing brightness & contrast etc.
3. Image Restoration: Image restoration is an area that also deals with improving the appearance of an image.
4. Color Image Processing: This may include color modeling and processing in a digital domain etc.
5. Wavelets and Multi-Resolution Processing: Wavelets are the foundation for representing images in various degrees of resolution.
6. Compression: Compression deals with techniques for reducing the storage required to save an image or the bandwidth to transmit it.
7. Morphological Processing: Morphological processing deals with tools for extracting image components that are useful in the representation and description of shape.

8. Segmentation: Segmentation procedures partition an image into its constituent parts or objects
9. Representation and Description: Representation and description almost always follow the output of a segmentation stage, which usually is raw pixel data, constituting either the boundary of a region or all the points in the region itself.
10. Object recognition: Recognition is the process that assigns a label, such as, “vehicle” to an object based on its descriptors.
11. Knowledge Base: Knowledge may be as simple as detailing regions of an image where the information of interest is known to be located, thus limiting the search that must be conducted in seeking that information.

Components of an Image Processing System

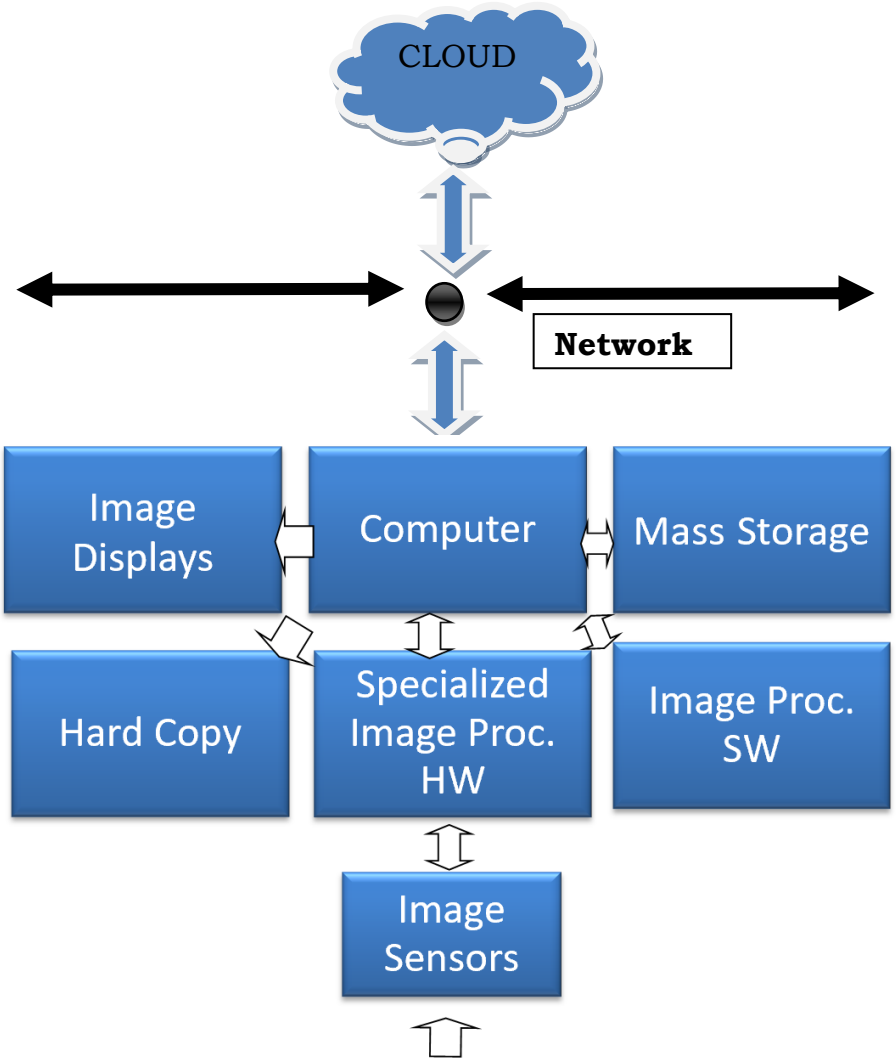


Figure: omponents of a General-Purpose Image Processing System

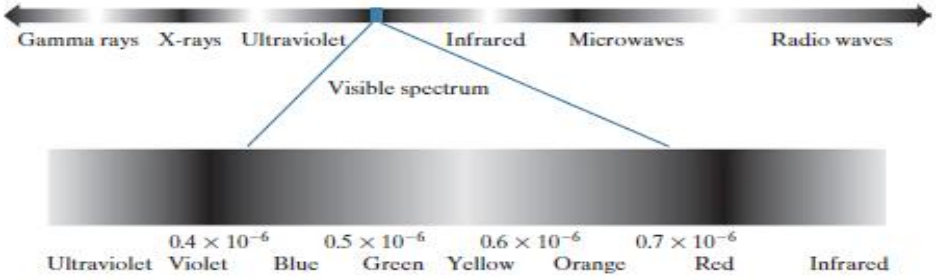
1. Image Sensors: With reference to sensing, two elements are required to acquire digital image. The first is a physical device that is sensitive to the energy radiated by the object we wish to image and second is specialized image processing hardware.
2. Specialize image processing hardware: It consists of the digitizer just mentioned, plus hardware that performs other primitive operations such as an arithmetic logic unit, which performs arithmetic such addition and subtraction and logical operations in parallel on images.
3. Computer: It is a general purpose computer and can range from a PC to a supercomputer depending on the application. In dedicated applications, sometimes specially designed computer are used to achieve the required level of performance
4. Software: It consists of specialized modules that perform specific tasks a well-designed package also includes capability for the user to write code, as a minimum, utilizes the specialized module. More sophisticated software packages allow the integration of these modules.
5. Mass storage – This capability is a must in image processing applications. An image of size 1024 x1024 pixels ,in which the intensity of each pixel is an 8- bit quantity requires one megabytes of storage space if the image is not compressed .Image processing applications falls into three principal categories of storage
    a. Short term storage for use during processing
    b. On line storage for relatively fast retrieval
    c. Archival storage such as magnetic tapes and disks
6. Image displays: Image displays in use today are mainly color TV monitors. These monitors are driven by the outputs of image and graphics displays cards that are an integral part of computer system
7. Hardcopy devices - The devices for recording image includes laser printers, film cameras, heat sensitive devices inkjet units and digital units such as optical and CD ROM disk. Films provide the highest possible resolution, but paper is the obvious medium of choice for written applications.
8. Networking: It is almost a default function in any computer system in use today because of the large amount of data inherent in image processing applications. The key consideration in image transmission bandwidth

Elements of Visual Perception

Light:

- Light is a particular type of electromagnetic radiation that can be sensed by the human eye.
- The visible band of the EM(electromagnetic) spectrum spans the range from 0.43 micrometers(violet) to about 0.79 micrometers(red)

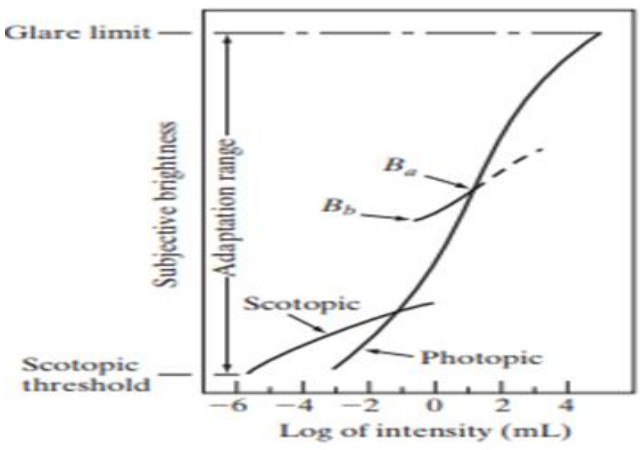
- The colors that human perceive in an object are determined by the nature of the light reflected from the object.
- A body that reflects light relatively balance in all visible wavelength appears white to the observer



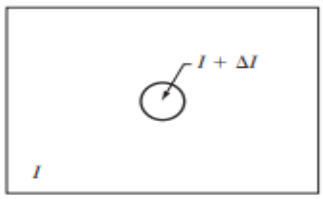
Brightness Adaptation and Discrimination

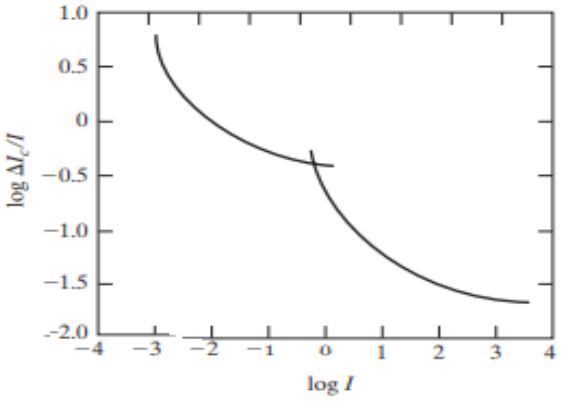
- Because digital images are displayed as sets of discrete intensities, the eye's ability to discriminate between different intensity levels is an important consideration in presenting image processing results.
- The range of light intensity levels to which the human visual system can adapt is enormous on the order of 10^{10} from the scotopic threshold to the glare limit. In photopic vision the range is about 10^{6}.candela per square metre (symbol: cd/m^{2})
- Scotopic vision is the vision of the eye under low light conditions.
- Photopic vision is the vision of the eye under well-lit conditions, normally usual daylight light intensity.
- Mesopic vision is the type of visions that takes place in intermediate lighting conditions.

- Subjective brightness(intensity as perceived by the HSV) is logarathimic function of the light intensity.
- The visual system doe not operate simultaneously over the 10^10 range.It accomplishes this large variation by changes in its overall sensitivity,a phenomena is known as brightness adaption



- The long solid curve represents the range of intensities to which the visual system can adapt.
- In photopic vision the range is about 10^6.
- The transition from scotopic to photopic vision is 10^6 . gradual over the approximate range from 0.001 to 0.1 millilambert (–3 to –1 mL in the log scale), as the double branches of the adaptation curve in this range show.
- The total range of distinct intensity levels it can discriminate simultaneously is rather small when compared with the total adaptation range. For any given set of conditions, the current sensitivity level of the visual system is called the brightness adaptation level, which may correspond, for example, to brightness Ba.
- The short intersecting curve represents the range of subjective brightness that the eye can perceive when adapted to this level. This range is rather restricted, having a level Bb at
  - The quantity ratio $\Delta I_c/I$, called weber ratio if where ΔI_c is more than 50% of the time with background illumination I.





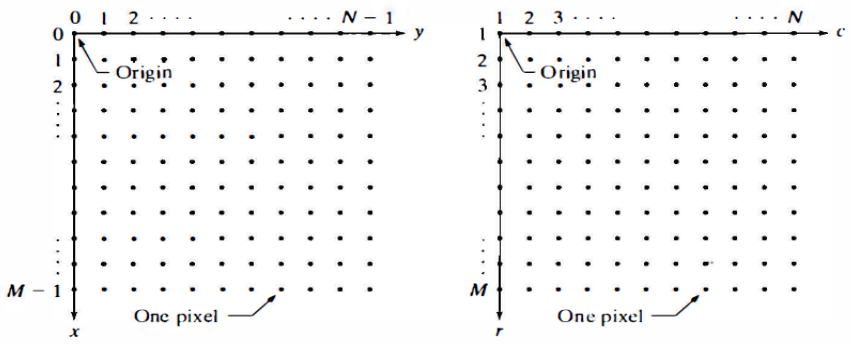
- A small value of $\Delta I_c/I$, means that a small percentage change in intensity is discriminable. This represents “good” brightness discrimination.
- Conversely, a large value of $\Delta I_c/I$, means that a large percentage change in intensity is required. This represents “poor” brightness discrimination.

Digital Image Representation-Image as 2D

An image may be defined as a two-dimensional function f(x, y), where x and y are spatial (plane) coordinates, and the amplitude off at any pair of coordinates is called the intensity of the image at that point. The term gray level is used often to refer to the intensity of monochrome images. Color images are formed by a combination of individual images. For example, in the RGB color system a color image consists of three individual monochrome images, referred to as the red (R), green (G), and blue (B) primary (or component) images. Converting such an image to digital form requires that the coordinates, as well as the amplitude, be digitized. Digitizing the coordinate values is called sampling; digitizing the amplitude values is called quantization.

Coordinate Conventions

The result of sampling and quantization is a matrix of real numbers. Assume that an image f(x, y) is sampled so that the resulting image has M rows and N columns. We say that the image is of size M X N. The values of the coordinates are discrete quantities. For notational clarity and convenience, we use integer values for these discrete coordinates. In many image processing books, the image origin is defined to be at (x, y) = (0, 0). The next coordinate values along the first row of the image are (x, y) = (0, 1). The notation (0, 1) is used to signify the second sample along the first row. Figure 2.1(a) shows this coordinate convention. Note that x ranges from 0 to M - 1 and y from 0 to N - 1 in integer increments. Image Processing Toolbox documentation refers to the coordinates in Fig.



a b

FIGURE 2.1 Coordinate conventions used (a) in many image processing books, and (b) in the Image Processing Toolbox.

Images as Matrices

The coordinate system in Fig.2.1 (a) and the preceding discussion lead to the following representation for a digitized image:

$$f(x,y) = \begin{bmatrix} f(0,0) & f(0,1) & \cdots & f(0,N-1) \\ f(1,0) & f(1,1) & \cdots & f(1,N-1) \\ \vdots & \vdots & & \vdots \\ f(M-1,0) & f(M-1,1) & \cdots & f(M-1,N-1) \end{bmatrix}$$

The right side of this equation is a digital image. Each element of this array is called an image element, picture element, pixel, or pet. The terms image and pixel are used throughout the rest of our discussions to denote a digital image and its elements.

A digital image can be represented in the form of matrix using Python as follows:

```
import numpy as np
from PIL import Image

# Open an image file
# Example: assuming 'example_image.jpg' exists in the same directory
try:
    img = Image.open('image_0.jpg')

    # Convert the PIL image object to a NumPy array (matrix)
    img_matrix = np.array(img)

    # Print the shape of the matrix (e.g., (height, width, channels))
    print(f"Image matrix shape: {img_matrix.shape}")

    # Print the data type of the pixels (usually uint8)
    print(f"Data type: {img_matrix.dtype}")

    # Print a small section of the matrix to show the data
    print("Top-left 3x3 pixel values:")
    print(img_matrix[:3, :3])

except FileNotFoundError:
    print("Please make sure 'image_0.jpg' is in the current directory.")
```

```
Image matrix shape: (2160, 3840, 3)
Data type: uint8
Top-left 3x3 pixel values:
[[[125 217 242]
  [143 233 255]
  [159 246 255]]

 [[  0  97 124]
  [ 55 150 178]
  [101 194 225]]

 [[  0  87 118]
  [  0  97 129]
  [  0  91 124]]]
```

READING and DISPLAYING IMAGES

1. **Using ImageIO**

ImageIO is used for reading and writing images in various formats like PNG, JPEG, GIF, TIFF and more. It's particularly useful for scientific and multi-dimensional image data likie medical images or animated GIFs.

```
import imageio.v2 as iio
import matplotlib.pyplot as plt
img = iio.imread("/image1.jpg")
plt.imshow(img)
```

2. **Using Matplotlib**

Matplotlib is a multi-platform data visualization library built on NumPy arrays and designed to work with the broader SciPy stack. It was introduced by John Hunter in the year 2002. Matplotlib comes with a wide variety of plots. Plots helps to understand trends, patterns and to make correlations

```
import matplotlib.image as mpimg
import matplotlib.pyplot as plt
img = mpimg.imread('/image1.jpg')
plt.imshow(img)
```

3. Using PIL

PIL is the Python Imaging Library which provides the python interpreter with image editing capabilities. Pillow is user friendly and easy to use library developed by Alex Clark and other contributors.

```
from PIL import Image
img = Image.open('Images/sanj5.jpeg')
img.show()
print(img.mode)
```

Writing Images:

Writing images in Python is typically handled using either the **OpenCV** or **Pillow (PIL)** libraries, both of which make the process straightforward by automatically determining the correct file encoding from the chosen file extension. If image is data structured as NumPy arrays—which is standard for computer vision and machine learning tasks—OpenCV's cv2.imwrite('filename.jpg', image_array) function is generally the most efficient and common choice. But for basic image manipulation and treating images as objects, the Pillow library allows you to easily save your work simply by calling image_object.save('filename.png').

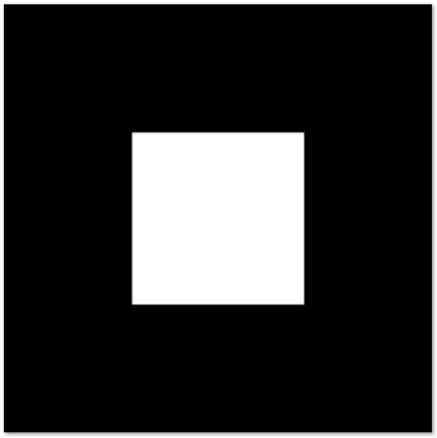
Example 1: Using OpenCV

OpenCV is the standard for computer vision. Remember that OpenCV loads images as NumPy arrays in **BGR** (Blue, Green, Red) format, not the standard RGB.

```
import cv2
import numpy as np

def main():
    # 1. First, we need an image to work with.
    # For this script we will create a simple black image with a white square
    # using numpy, so you don't need a pre-existing image file to test this code.
    print("Creating a sample image...")
```

```
    # Create a completely black image (500x500 pixels, 3 color channels)
    image = np.zeros((500, 500, 3), dtype=np.uint8)

    # Draw a white rectangle in the middle to make it interesting
    # Top-left corner (150, 150) to bottom-right corner (350, 350)
    cv2.rectangle(image, (150, 150), (350, 350), (255, 255, 255), thickness=-1)

    # 2. Define the filename where we want to save our image
    output_filename = "saved_sample_image.jpg"

    # 3. Use cv2.imwrite() to save the image to disk
    # This function returns True if the image was successfully saved
    success = cv2.imwrite(output_filename, image)

    if success:
        print(f"Success! The image was saved to the current directory as '{output_filename}'")
    else:
        print("Error: Could not save the image. Please check your permissions and file path.")

if __name__ == "__main__":
    main()
```

Output:



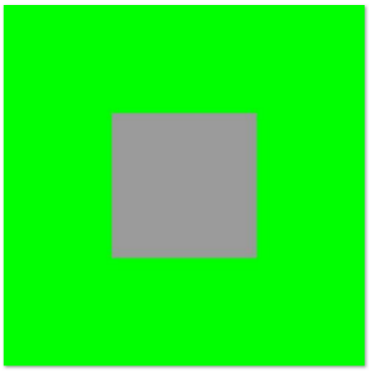
Example 2: Using Pillow (PIL)

Pillow is incredibly user-friendly for general image manipulation and graphic design tasks. It loads images as Image objects in standard **RGB** format.

```
from PIL import Image, ImageDraw

def main():
    print("Creating a sample image using Pillow...")

    # 1. Create a new image using Image.new()
    # Mode 'RGB' means 3 color channels (Red, Green, Blue)
```

```
    # The size is a tuple (width, height) - in this case 500x500
    # The initial background color is green (0, 255, 0)
    image = Image.new('RGB', (500, 500), color=(0, 255, 0))

    # 2. To draw on the image, we create an ImageDraw object
    draw = ImageDraw.Draw(image)

    # Draw a gray rectangle in the middle
    # Coordinates are [x0, y0, x1, y1] for top-left and bottom-right
    draw.rectangle([150, 150, 350, 350], fill=(155, 155, 155))

    # 3. Define the filename where we want to save our image
    output_filename = "saved_sample_image_pil.jpg"

    # 4. Use the save() method on the Image object to save it to disk
    try:
        image.save(output_filename)
            print(f'Success! The image was saved to the current directory as '{output_filename}'")
    except Exception as e:
        print(f'Error: Could not save the image. Exception details: {e}")

if __name__ == "__main__":
    main()
```

Output:



Gray-scale Images

- A gray-scale image is a data matrix whose values represent shades of gray without any apparent color.
- The darkest possible shade is black, which is the total absence of transmitted or reflected light. The lightest possible shade is white, the total transmission or reflection of light at all visible wavelength s.

Name	Description
double	Double-precision, floating-point numbers in the approximate range $\pm 10^{308}$ (8 bytes per element).
single	Single-precision floating-point numbers with values in the approximate range $\pm 10^{38}$ (4 bytes per element).
uint8	Unsigned 8-bit integers in the range [0, 255] (1 byte per element).
uint16	Unsigned 16-bit integers in the range [0, 65535] (2 bytes per element).
uint32	Unsigned 32-bit integers in the range [0, 4294967295] (4 bytes per element).
int8	Signed 8-bit integers in the range [−128, 127] (1 byte per element).
int16	Signed 16-bit integers in the range [−32768, 32767] (2 bytes per element).
int32	Signed 32-bit integers in the range [−2147483648, 2147483647] (4 bytes per element).
char	Characters (2 bytes per element).
logical	Values are 0 or 1 (1 byte per element).

Color Image Representation

The Image Processing Toolbox handles color images either as indexed images or RGB (red, green, blue) images.

RGB Images

An RGB color image is an M X N X 3 array of color pixels, where each color pixel is a triplet corresponding to the red, green, and blue components of an RGB image at a specific spatial location (see Fig. 7.1). An RGB image may be viewed as a "stack" of three gray-scale images that, when fed into the red, green, and blue inputs of a color monitor, produce a color image on the screen. By convention, the three images forming an RGB color image are referred to as the red, green, and blue component images.

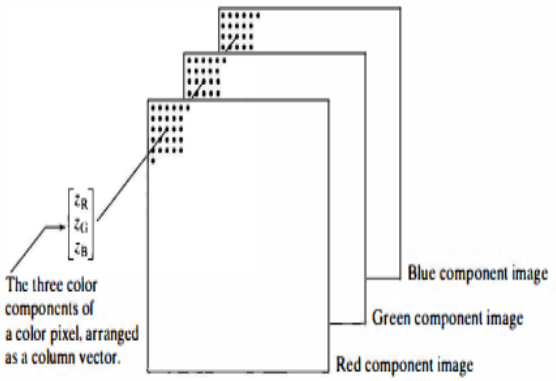


FIGURE 7.1 Schematic showing how pixels of an RGB color image are formed from the corresponding pixels of the three component images.

Indexed Images

An indexed image has two components:

- A data matrix of integers X
- Color map matrix map.

Matrix map is an m X 3 array of class double containing floating-point values in the range (0, l). The length of the map is equal to the number of colors it defines. Each row of map specifies the red, green, and blue components of a single color (if the three columns of map are equal, the color map becomes a gray-scale map). An indexed image uses "direct mapping" of pixel intensity values to color-map values. The color of each pixel is determined by using the corresponding value of integer matrix X as an index (hence the name indexed image) into map.

These concepts are illustrated in Fig. 7.3.

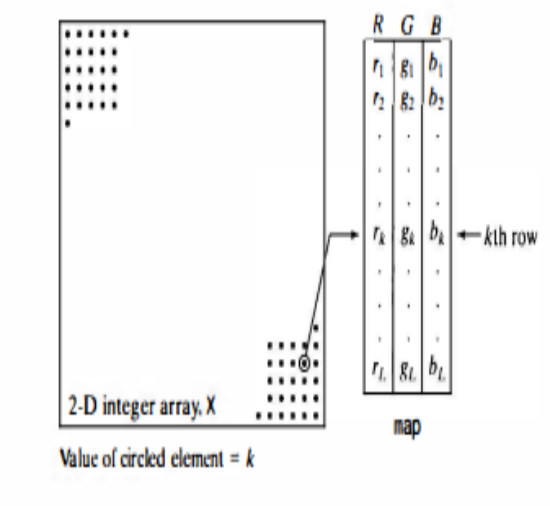
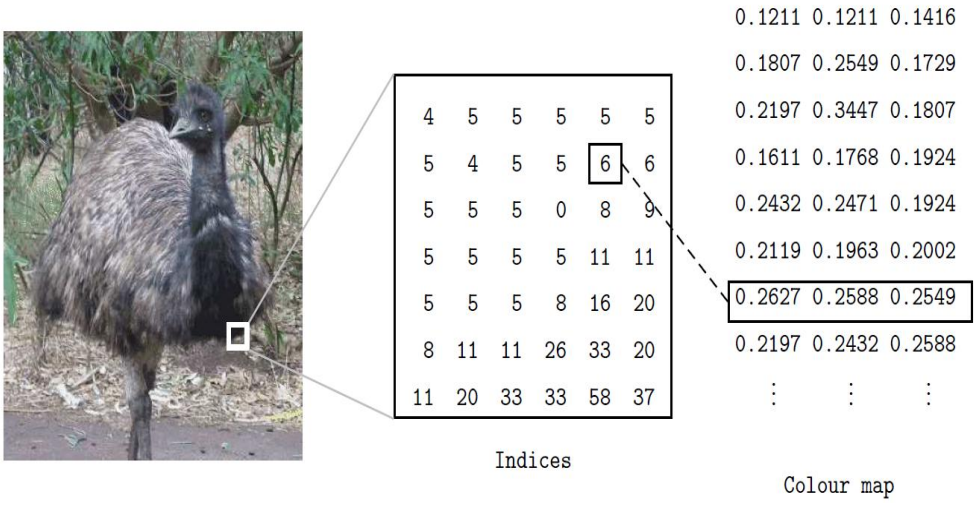


FIGURE 7.3 Elements of an indexed image. The value of an element of integer array X determines the row number in the color map. Each row contains an RGB triplet, and *L* is the total number of rows.



How to Use the HSV Color Model

Hue

Hue is the color portion of the model, expressed as a number from 0 to 360 degrees:

Red falls between 0 and 60 degrees.

Yellow falls between 61 and 120 degrees.

Green falls between 121 and 180 degrees.

Cyan falls between 181 and 240 degrees.

Blue falls between 241 and 300 degrees.

Magenta falls between 301 and 360 degrees.

Saturation

Saturation describes the amount of gray in a particular color, from 0 to 100 percent.

Value (or Brightness)

Value works in conjunction with saturation and describes the brightness or intensity of the color, from 0 to 100 percent, where 0 is completely black, and 100 is the brightest and reveals the most color.

What is YCbCr ?
Here Y is the luma component of the color. Luma components are the brightness of the color.

Convert RGB images to gray images

```
import cv2
import os

def main():
    print("Converting RGB image to Grayscale using OpenCV...")

    # 1. Use the local image specified
    input_path = r"E:\Professional Files\4th sem BCA\rosecolor.jpg"
    if not os.path.exists(input_path):
        print(f"Error: Could not find image at {input_path}")
        return
    # 2. Read the image
    # cv2.imread loads images in BGR format by default
    image = cv2.imread(input_path)
      if image is None:
        print("Error: Could not read image.")
        return
    # 3. Convert the image from BGR (OpenCV's default RGB order) to Grayscale
    # It uses the formula: Y = 0.299 R + 0.587 G + 0.114 B
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    # 4. Save the new grayscale image
    output_filename = "output_grayscale.png"
    cv2.imwrite(output_filename, gray_image)
```

```
    print(f"Process Complete!")
    print(f"Original image saved as: {input_path}")
    print(f"Grayscale image saved as: {output_filename}")

if __name__ == "__main__":
    main()
```

Result:

Input Image

Output image

Basic Operations on Images:

Cropping:

In Digital Image processing, **Image Cropping** is the process of selecting a specific **Region of Interest (ROI)** and discarding the data outside that area. Mathematically, this is a coordinate-based sub-sampling process where you define a new bounding box within the original image matrix.

The Core Concepts

- **Spatial Sub-setting:** Every digital image is a 2D grid (or 3D if you include color channels). Cropping is essentially creating a new matrix I_{new} from the original I_{old} by selecting a range of rows and columns.

  $$I_{new}(x,y) = I_{old}(x_0+x,\ y_0+y)$$

  where (x_0, y_0) is the top-left corner of your desired crop.
- **Matrix Slicing:** In Python (using NumPy/OpenCV), this is handled via **slicing**. Because images are stored as arrays, you simply tell the computer which "index range" you want to keep. Unlike resizing, cropping does not change the pixel density or interpolate new data; it purely subtracts information.

Python Implementation (OpenCV vs. Pillow)

Since OpenCV treats images as NumPy arrays, cropping is as simple as slicing a list. Pillow uses a specific .crop() method that defines the box as a tuple.

Using OpenCV (NumPy Slicing)

In OpenCV, the syntax is image[y_start:y_end, x_start:x_end]. Note that **y (rows)** comes before **x (columns)**.

```
import cv2
img = cv2.imread('input.jpg')
# Crop from y=100 to 400, and x=200 to 500
cropped_img = img[100:400, 200:500]
cv2.imwrite('opencv_crop.jpg', cropped_img)
```

Using Pillow

Pillow uses a (left, top, right, bottom) coordinate system.

```
from PIL import Image
with Image.open('input.jpg') as img:
    # Define the box: (x_start, y_start, x_end, y_end)
    crop_box = (200, 100, 500, 400)
    cropped_img = img.crop(crop_box)
    cropped_img.save('pillow_crop.jpg')
```

```
import cv2
import os
def main():
    print("Cropping an image using OpenCV array slicing...")
    # 1. Define the input image path
    input_path = r"E:\Professional Files\4th sem BCA\rosecolor.jpg"
    # 2. Check if the file exists
    if not os.path.exists(input_path):
        print(f"Error: Could not find image at '{input_path}'")
        return
    # 3. Read the image
    image = cv2.imread(input_path)
    if image is None:
        print("Error: Could not read the image. It might be corrupted or in an unsupported format.")
        return
    # Print original dimensions
    height, width, channels = image.shape
    print(f"Original image dimensions: {width}x{height} pixels (Width x Height)")
    # 4. Define the region to crop (Region of Interest - ROI)
    # The syntax for cropping an image in OpenCV is:
    # cropped_image = original_image[start_y:end_y, start_x:end_x]
    # Example: Let's crop to a central region of the image
    # We will compute a region that is half the size of the original, centered.
    start_y = height // 4
```

```
    end_y = start_y + (height // 2)
    start_x = width // 4
    end_x = start_x + (width // 2)
    print(f"Cropping region: Y=[{start_y}:{end_y}], X=[{start_x}:{end_x}]")
    # 5. Perform the array slicing to crop the image
    cropped_image = image[start_y:end_y, start_x:end_x]
    # Print new dimensions
    new_height, new_width, _ = cropped_image.shape
    print(f"Cropped image dimensions: {new_width}x{new_height} pixels")
    # 6. Save the cropped image to disk
    output_filename = "output_cropped.jpg"
    success = cv2.imwrite(output_filename, cropped_image)
    if success:
        print(f"Success! The cropped image was saved to the current directory as '{output_filename}'")
    else:
        print("Error: Could not save the cropped image.")
if __name__ == "__main__":
    main()
```

Result:

Input Image

Cropped Image

Complementing:

In Digital Image Processing, **Complementing** an image (also known as **Inverting**) is a point-processing operation where the intensity value of each pixel is subtracted from the maximum possible intensity value of the image's dynamic range. For a standard 8-bit grayscale image, the maximum value is 255.

The Core Concepts

- **Mathematical Transformation:** The operation is a simple linear mapping. If I(x,y) is the original pixel value and $I^c(x, y)$ is the complemented value:

$$I^c(x, y) = L - 1 - I(x, y)$$

For 8-bit images (L=256), the formula is:

$$I^c(x, y) = 255 - I(x, y)$$

- **Visual Effect:** Black pixels (0) become white (255), and white pixels (255) become black (0). In colour images, each channel (Red, Green, and Blue) is inverted independently, which results in "photographic negative" colours (e.g., Red becomes Cyan).

Python Implementation

1. Using OpenCV (and NumPy)

Since images in OpenCV are NumPy arrays, you can use the bitwise NOT operator or simple subtraction. NumPy's vectorized operations make this extremely fast.

```
import cv2
img = cv2.imread('input.jpg')
# Option A: Using bitwise NOT (most common)
inverted_img = cv2.bitwise_not(img)
# Option B: Using simple subtraction (mathematical approach)
# inverted_img = 255 - img
cv2.imwrite('opencv_inverted.jpg', inverted_img)
```

2. Using Pillow

Pillow provides a dedicated module called ImageChops (Image Channel Operations) for these types of mathematical shifts.

```
from PIL import Image, ImageChops
with Image.open('input.jpg') as img:
    # Use the invert function from ImageChops
    inverted_img = ImageChops.invert(img)
    inverted_img.save('pillow_inverted.jpg')
```

Why Complement?

1. **Enhancing Detail:** Inverting an image can make features hidden in dark regions more visible to the human eye, which is common in **Medical Imaging** (like X-rays or CT scans).
2. **Threshold Preparation:** Some segmentation algorithms expect the object of interest to be light on a dark background. If your original image is dark-on-light, you must complement it first.
3. **Visual Graphics:** Creating negatives for artistic effects or UI "Dark Mode" transitions.

```
import cv2
import os

def main():
    print("Generating the complement (negative) of an image...")
    # 1. Define the local image path
    input_path = r"E:\Professional Files\4th sem BCA\rosecolor.jpg"
    # 2. Check if the file exists
    if not os.path.exists(input_path):
        print(f"Error: Could not find image at '{input_path}'")
        return
   # 3. Read the image
    image = cv2.imread(input_path)
    if image is None:
         print("Error: Could not read the image. It might be corrupted or in an unsupported format.")
        return
    # 4. Generate the complement of the image
    # The complement (or negative) of an image is achieved by subtracting each pixel value from 255.
    # We can do this using cv2.bitwise_not() which is highly optimized for this exact operation.
    complement_image = cv2.bitwise_not(image)
    # Alternatively, you could also use numpy math:
    # complement_image = 255 - image
    # 5. Save the complement image to disk
    output_filename = "output_complement.jpg"
    success = cv2.imwrite(output_filename, complement_image)
    if success:
        print(f"Success! The complement image was saved to the current directory as '{output_filename}'")
    else:
        print("Error: Could not save the complement image.")
if __name__ == "__main__":
    main()
```

Results:

Input Image

Complement Image

Resizing:

Resizing (or **Image Scaling**) is a geometric transformation that changes the total number of pixels in an image. Unlike cropping, which simply discards data, resizing involves **Image Interpolation** to estimate new pixel values when an image is stretched or compressed.

The Core Concepts

- **Spatial Transformation:** Resizing maps a coordinate (x,y) in the original image to a new coordinate (x', y') in the target image using a scaling factor S.

$$x' = S_x.x, \text{ and } y' = S_y.y$$

- **Interpolation Algorithms:** When you "upscale" an image, the computer must "invent" pixels between the existing ones. When you "downscale," it must intelligently merge pixels. Common methods include:
  - **Nearest Neighbour:** Takes the value of the closest pixel. Fast but results in "blocky" or "pixelated" images.
  - **Bilinear Interpolation:** Calculates the new pixel value based on a weighted average of the 2x2 neighbourhood of pixels. It is smoother than Nearest Neighbour.
  - **Bicubic Interpolation:** Uses a 4x4 neighbourhood. It is slower but produces much sharper results with fewer artifacts, making it the standard for high-quality upscaling.

Python Implementation

1. Using OpenCV (cv2.resize)

OpenCV allows you to resize by specifying absolute dimensions or by using scaling factors.

```
import cv2
img = cv2.imread('input.jpg')
# Option A: Resize to specific dimensions (Width, Height)
resized_dims = cv2.resize(img, (800, 600), interpolation=cv2.INTER_CUBIC)
# Option B: Resize by scaling factor (e.g., 50% of original size)
scaled_img=cv2.resize(img,None,fx=0.5,fy=0.5,interpolation=cv2.INTER_LINEAR
)
cv2.imwrite('opencv_resized.jpg', scaled_img)
```

2. Using Pillow (img.resize)

Pillow provides a very clean interface and several high-quality filters like LANCZOS for downsampling.

```
from PIL import Image
with image.open('input.jpg') as img:
    # Define new size as a (width, height) tuple
    new_size = (800, 600)
     # Resizing with a high-quality filter
```

```
resized_img = img.resize(new_size, Image.Resampling.LANCZOS)
resized_img.save('pillow_resized.jpg')
```

Why Resize?

1. **Normalization:** Neural networks usually require a fixed input size (e.g., 224x224 pixels).
2. **Performance:** Reducing the resolution of high-definition images significantly speeds up processing time for filters and edge detection.
3. **Storage/Bandwidth:** Downscaling images is essential for web optimization to ensure fast loading times.

```
import cv2
import os

def main():
    print("Resizing an image using OpenCV...")
    # 1. Define the local image path
    input_path = r"E:\Professional Files\4th sem BCA\rosecolor.jpg"
    # 2. Check if the file exists
    if not os.path.exists(input_path):
        print(f"Error: Could not find image at '{input_path}'")
        return
    # 3. Read the image
    image = cv2.imread(input_path)
    if image is None:
        print("Error: Could not read the image. It might be corrupted or in an unsupported format.")
        return
    # Print original dimensions
    height, width = image.shape[:2]
    print(f"Original image dimensions: {width}x{height} pixels (Width x Height)")
    # 4. Resize the image
    # Method A: Providing exact new pixel dimensions
    # Let's say we want to resize it to exactly 800x600 px
    # new_dimensions = (800, 600)
    #resized_exact=cv2.resize(image,new_dimensions, interpolation=cv2.INTER_AREA)
    # Method B: Scaling it by a percentage (better for maintaining aspect ratio)
    # Let's scale it down by 50% (0.5 on x and y axes)
    scale_percent = 50
    new_width = int(width * scale_percent / 100)
    new_height = int(height * scale_percent / 100)
    new_dimensions = (new_width, new_height)
```

```
    # cv2.INTER_AREA is highly recommended for shrinking an image.
    # If zooming/enlarging, use cv2.INTER_CUBIC or cv2.INTER_LINEAR.
    resized_image=cv2.resize(image,new_dimensions,
interpolation=cv2.INTER_AREA)
    # Print new dimensions
    print(f"Resized image dimensions: {new_width}x{new_height} pixels (Scaled to
50%)")
    # 5. Save the resized image to disk
    output_filename = "output_resized.jpg"
    success = cv2.imwrite(output_filename, resized_image)
    if success:
        print(f"Success! The resized image was saved to the current directory as
'{output_filename}'")
    else:
        print("Error: Could not save the resized image.")

if __name__ == "__main__":
    main()
```

Result:

Input Image

Resized Image

Image Sampling and Quantization

To become suitable for digital processing, an image function f(x,y) must be digitized both spatially and in amplitude. Typically, a frame grabber or digitizer is used to sample and quantize the analog video signal. Hence in order to create an image which is digital, we need to convert continuous data into digital form. There are two steps in which it is done:

1. Sampling
2. Quantization

- Digitizing the coordinate values is called sampling.
- Digitizing the amplitude values is called quantization.

Some Basic Relationships Between Pixels

Neighbors of a Pixel

A pixel p at coordinates (x, y) has four horizontal and vertical neighbors whose coordinates are given by

$$(x+1, y), (x-1, y), (x, y+1), (x, y-1)$$

This set of pixels, called the 4-neighbors of p, is denoted by $N_4(p)$. Each pixel is a unit distance from (x, y), and some of the neighbors of p lie outside the digital image if (x, y) is on the border of the image. The four diagonal neighbors of p have coordinates (x+1, y+1), (x+1, y-1), (x-1, y+1), (x-1, y-1) and are denoted by $N_D(p)$. These points, together with the 4-neighbors, are called the 8-neighbors of p, denoted by $N_8(p)$. As before, some of the points in $N_D(p)$ and $N_8(p)$ fall outside the image if (x, y) is on the border of the image.

Adjacency, Connectivity, Regions, and Boundaries

- Pixel connectivity is a fundamental concept in digital image processing.
- It helps define important image features such as regions and boundaries.
- Two pixels are considered connected only if:
  - They are neighbors, and
  - Their gray levels satisfy a similarity criterion (e.g., equal intensity).
- In a binary image (values 0 and 1), pixels may be 4-neighbors, but they are connected only if they have the same value.

Let V be the set of gray-level values used to define adjacency. In a binary image, V={1} if we are referring to adjacency of pixels with value 1.In a gray scale image, the idea is the same, but set V typically contains more elements. For example, in the adjacency of pixels with a range of possible gray-level values 0 to 255, set V could be any subset of these 256 values. We consider three types of adjacency

We consider three types of adjacencies:

(a) 4-adjacency. Two pixels p and q with values from V are 4-adjacent if q is in the set $N_4(p)$.

(b) 8-adjacency. Two pixels p and q with values from V are 8-adjacent if q is in the set $N_8(p)$.

(c) m-adjacency (mixed adjacency).Two pixels p and q with values from V are m-adjacent if

(i) q is in $N_4(p)$, or

(ii) q is in $N_D(p)$ and the set $N_4(p) \cap N_4(q)$ has no pixels whose value are from V.

****